

第3回 AIプログラミング1

条件文

大西朔永

if文の基礎

第2章の復習

比較演算子

`==`, `!=`, `<`, `>`, `<=`, `>=`

2つの値を比較

bool型

`True` (真) / `False`

(偽)

比較演算子の結果

```
>>> 10 > 5
```

```
True
```

```
>>> 3 == 7
```

```
False
```

比較の結果 (`True` / `False`) を使って処理を分岐する

条件分岐とは

- 条件に応じて実行する処理を切り替える仕組み

日常の例

信号が青 → 渡る
信号が赤 → 止まる

プログラムの例

点数が60以上 → 合格と
表示
点数が60未満 → 不合格
と表示

Pythonでは **if文** で条件分岐を実現

if文の構文

if文

```
if 条件式:  
    ... 処理 (インデント)
```

- 条件式が `True` のとき、インデントされたブロックを実行
- 条件式が `False` のとき、ブロックをスキップ

例

```
if score >= 60:  
    print("合格")
```

C言語との比較：if文

C言語

```
if (score >= 60) {  
    ...printf("合格\n");  
}
```

- `()` で条件を囲む
- `{ }` でブロックを囲む

Python

```
if score >= 60:  
    ...print("合格")
```

- `()` 不要
- インデントでブロックを表現

Pythonはインデントがブロック構造を決定（`{ }` の代わり）

はじめてのif文

ai_programming_1_03_01.py

```
1  # 正の数かどうかを判定して表示するプログラム
2
3  # 整数を入力
4  number = int(input("整数を入力してください: "))
5
6  # 正の数かどうかを判定
7  if number > 0:
8      print(f"{number}は正の数です。")
```

実行結果（正の数）

整数を入力してください: 5
5は正の数です。

実行結果（0以下）

整数を入力してください: -3

- `input()` + `int()` で整数を入力
- 条件が `False`（0以下）のとき何も表示されない

インデントの重要性

正しいインデント

```
if number > 0:  
    print("正の数")  
    print("処理完了")
```

2行とも `if` ブロック内

インデントなし

```
if number > 0:  
    print("正の数")  
print("処理完了")
```

"処理完了" は常に実行

インデントエラーの例

```
if number > 0:  
print("正の数")...# IndentationError!
```

インデントは半角スペース
4つが標準

まとめ

条件分岐

条件に応じて
処理を切り替える

if文

```
if 条件式:  
True のとき実行
```

インデント

半角スペース4つ
ブロック構造を
決定

C言語の `{ }` に代わり、
Pythonはインデントでブロックを表す

if-else文 と if-elif-else文

if-else文の構文

if-else文

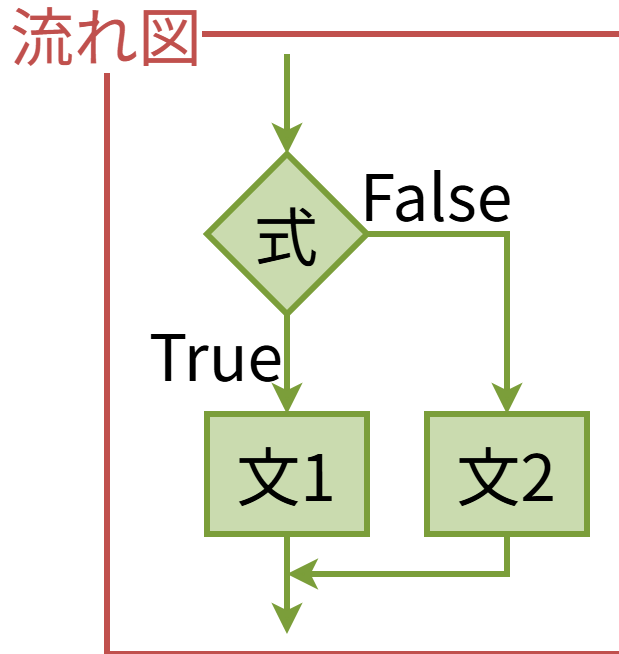
```
if 条件式:  
    ... 文1 (Trueのとき)  
else:  
    ... 文2 (Falseのとき)
```

例

```
if age >= 18:  
    ... print("成人")  
else:  
    ... print("未成年")
```

- 条件が True → 文1、False → 文2
- 必ずどちらかが実行される (2分岐)

if-else文の処理フロー



偶数・奇数の判定

ai_programming_1_03_02.py

```
1  # 偶数・奇数を判定して表示するプログラム
2
3  # 整数を入力
4  number = int(input("整数を入力してください: "))
5
6  # 偶数・奇数を判定
7  if number % 2 == 0:
8      print(f"{number}は偶数です。")
9  else:
10     print(f"{number}は奇数です。")
```

実行結果（偶数）

整数を入力してください: 8
8は偶数です。

実行結果（奇数）

整数を入力してください: 7
7は奇数です。

- % 演算子で2で割った余りを計算
- 余りが0 → 偶数、それ以外 → 奇数

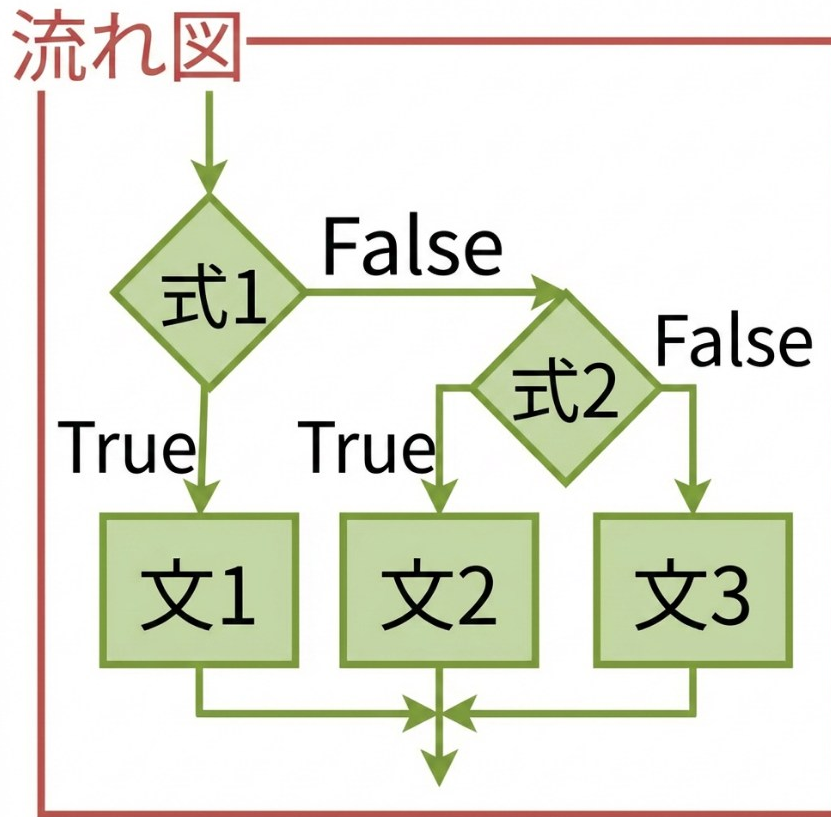
if-elif-else文の構文

if-elif-else文

```
if 条件式1:  
    ... 文1  
elif 条件式2:  
    ... 文2  
else:  
    ... 文3
```

- `elif` は複数記述可能（3つ以上の分岐に対応）
- 上から順に評価し、最初に `True` になった処理を実行

if-elif-else文の処理フロー



成績判定プログラム

ai_programming_1_03_03.py

```
1  # 点数からA/B/C/Dの成績を判定するプログラム
2
3  # 点数を入力
4  score = int(input("点数を入力してください: "))
5
6  # 成績判定
7  if score >= 90:
8      grade = "A"
9  elif score >= 70:
10     grade = "B"
11 elif score >= 50:
12     grade = "C"
13 else:
14     grade = "D"
15
```

実行結果 (A判定)

点数を入力してください: 95
成績はAです。

実行結果 (C判定)

点数を入力してください: 55
成績はCです。

- 90以上→A、70以上→B、50以上→C、それ以外→D
- 上から順に評価されるため、条件は大きい順に記述

成績判定プログラム

ai_programming_1_03_03.py

```
1
2
3  # 点数を入力
4  score = int(input("点数を入力してください: "))
5
6  # 成績判定
7  if score >= 90:
8      grade = "A"
9  elif score >= 70:
10     grade = "B"
11 elif score >= 50:
12     grade = "C"
13 else:
14     grade = "D"
15
16 print(f"成績は{grade}です。")
```

実行結果 (A判定)

点数を入力してください: 95
成績はAです。

実行結果 (C判定)

点数を入力してください: 55
成績はCです。

- 90以上→A、70以上→B、50以上→C、それ以外→D
- 上から順に評価されるため、条件は大きい順に記述

C言語との比較：else if vs elif

C言語

```
if (score >= 90) {  
    grade = 'A';  
} else if (score >= 70) {  
    grade = 'B';  
} else {  
    grade = 'C';  
}
```

■ else if (2語)

Python

```
if score >= 90:  
    grade = "A"  
elif score >= 70:  
    grade = "B"  
else:  
    grade = "C"
```

■ elif (1語)

C言語の else if = Pythonの elif

まとめ

if-else文

2分岐

必ずどちらかを
実行

if-elif-else文

多分岐

`elif` は複数記
述可能

評価順序

上から順に評価
最初の `True` を
実行

分岐の数に応じて `if` → `if-else` → `if-elif-else` を使
い分ける

論理演算子

論理演算子とは

- 複数の条件を組み合わせる演算子
- 条件分岐をより柔軟に記述可能

日常の例

年齢が18歳以上 **かつ** 学生
→ 学割適用

プログラムの例

```
age >= 18 and  
is_student
```

`and`、`or`、`not` の3種類

and演算子

and演算子

条件1 **and** 条件2

```
>>> 10 > 5 and 3 < 7
True
>>> 10 > 5 and 3 > 7
False
```

条件1	条件2	結果
True	True	True
True	False	False
False	True	False
False	False	False

- 両方の条件が True のときだけ True

or演算子

or演算子

条件1 · or · 条件2

```
>>> 10 > 5 or 3 > 7
True
>>> 10 < 5 or 3 > 7
False
```

条件1	条件2	結果
True	True	True
True	False	True
False	True	True
False	False	False

- いずれかの条件が True のとき True

not演算子

not演算子

not · 条件

- 条件の真偽を反転 (True ↔ False)

条件	結果
True	False
False	True

```
>>> not True
False
>>> not (10 > 5)
False
>>> not (3 > 7)
True
```


C言語との比較：論理演算子

C言語

```
if (a > 0 && b > 0) { ... }  
if (a > 0 || b > 0) { ... }  
if (!flag) { ... }
```

- `&&` (AND)
- `||` (OR)
- `!` (NOT)

Python

```
if a > 0 and b > 0: ...  
if a > 0 or b > 0: ...  
if not flag: ...
```

- `and` (AND)
- `or` (OR)
- `not` (NOT)

Pythonは**英単語**で論理演算子を記述（可読性が高い）

範囲判定プログラム

ai_programming_1_03_04.py

```
1  # 数値が範囲内かどうかを判定するプログラム
2
3  # 整数を入力
4  number = int(input("整数を入力してください: "))
5
6  # 1以上100以下かどうかを判定
7  if number >= 1 and number <= 100:
8      print(f"{number}は1以上100以下の範囲内です。")
9  else:
10     print(f"{number}は範囲外です。")
```

実行結果（範囲内）

整数を入力してください: 50
50は1以上100以下の範囲内です。

実行結果（範囲外）

整数を入力してください: 150
150は範囲外です。

- `and` で2つの条件を同時に判定
- `number >= 1 and number <= 100` で範囲チェック

まとめ

and

両方 True

→ True

「かつ」

or

いずれか True

→ True

「または」

not

真偽を反転

「でない」

論理演算子で**複合条件**を組み立て、柔軟な条件分岐を実現

条件文のネストと応用

ネストしたif文

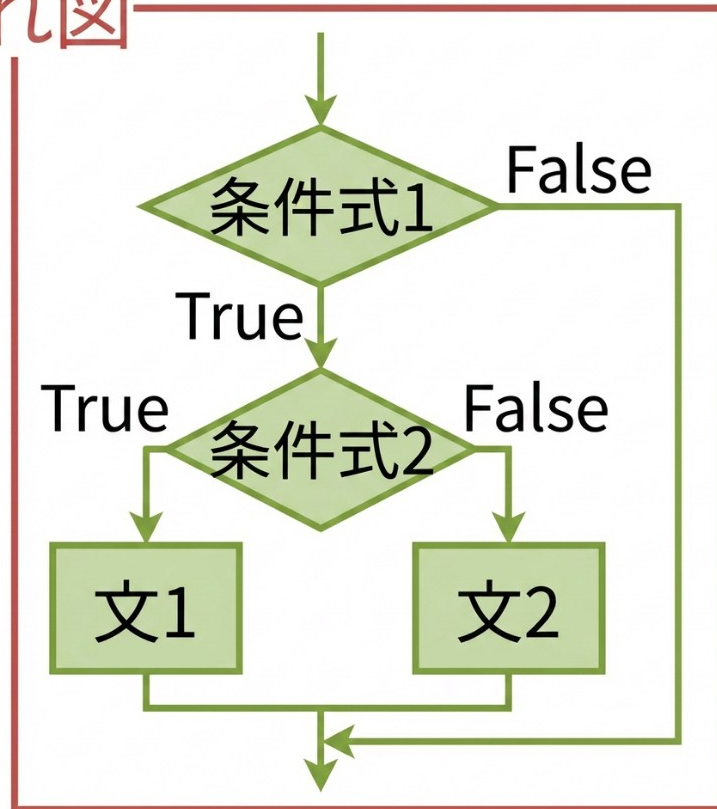
ネストしたif文

```
if 条件式1:  
    ... if 条件式2:  
        ... 文1  
    ... else:  
        ... 文2
```

- if文の中にif文を書く → ネスト（入れ子）
- インデントが1段階深くなる（4 + 4 = 8スペース）

ネストの処理フロー

流れ図



ネストを使った判定

ai_programming_1_03_05.py

```
1  # 正負判定→偶数・奇数判定をネストで実装するプログラム
2
3  # 整数を入力
4  number = int(input("整数を入力してください: "))
5
6  # 正負の判定 (外側のif文)
7  if number > 0:
8      # 偶数・奇数の判定 (内側のif文)
9      if number % 2 == 0:
10         print(f"{number}は正の偶数です。")
11     else:
12         print(f"{number}は正の奇数です。")
13 elif number < 0:
14     if number % 2 == 0:
15         print(f"{number}は負の偶数です。")
16     else:
17         print(f"{number}は負の奇数です。")
18 else:
19     print("0を入力しました。")
```

実行結果 (正の偶数)

整数を入力してください: 8
8は正の偶数です。

実行結果 (負の奇数)

整数を入力してください: -5
-5は負の奇数です。

- 外側: 正・負・ゼロの判定
- 内側: 偶数・奇数の判定

ネストを使った判定

ai_programming_1_03_05.py

```
6  # 正負の判定 (外側のif文)
7  if number > 0:
8      # 偶数・奇数の判定 (内側のif文)
9      if number % 2 == 0:
10         print(f"{number}は正の偶数です。")
11     else:
12         print(f"{number}は正の奇数です。")
13 elif number < 0:
14     if number % 2 == 0:
15         print(f"{number}は負の偶数です。")
16     else:
17         print(f"{number}は負の奇数です。")
18 else:
19     print("0です。")
```

実行結果 (正の偶数)

整数を入力してください:
8は正の偶数です。↵

実行結果 (負の奇数)

整数を入力してください:
-5は負の奇数です。↵

- 外側: 正・負・ゼロの判定
- 内側: 偶数・奇数の判定

in演算子

in演算子

値・in・リスト

- 値がリストに含まれているかを判定 (True / False)
- リストの詳細は第5章で学ぶ

例

```
>>> color = "赤"
>>> color in ["赤", "青", "緑"]
True
>>> "黄" in ["赤", "青", "緑"]
False
```

条件式（三項演算子）

条件式（三項演算子）

値1・if・条件・else・値2

- 条件が True → 値1、False → 値2
- 1行で条件分岐を記述可能

C言語

```
int max = (a > b) ? a : b;
```

Python

```
max_val = a if a > b else b
```

ネストの整理

- ネストが深くなると可読性が低下

ネストが深い例

```
if a > 0:  
    if b > 0:  
        if c > 0:  
            print("全て正")
```

3段階のネスト

論理演算子で整理

```
if a > 0 and b > 0 and c > 0:  
    print("全て正")
```

1段階で同じ結果

elifや論理演算子でネストを浅くすることを意識する

まとめ

ネスト

if文の中にif文
段階的な条件判定

in演算子

値の所属判定

```
値 in リスト
```

条件式

1行で分岐

```
値1 if 条件  
else 値2
```

ネストは便利だが、
深くなりすぎないように論理演算子で整理する

条件文の総合演習

BMI判定プログラム

ai_programming_1_03_06.py

```
1  # BMIを計算し判定結果を表示するプログラム
2
3  # 身長(cm)と体重(kg)を入力
4  height_cm = float(input("身長(cm)を入力してください: "))
5  weight = float(input("体重(kg)を入力してください: "))
6
7  # BMIを計算 (身長をmに変換)
8  height_m = height_cm / 100
9  bmi = weight / (height_m**2)
10
11 # BMIの判定
12 if bmi < 18.5:
13     result = "低体重"
14 elif bmi < 25.0:
15     result = "普通体重"
```

実行結果

身長(cm)を入力してください: 170
体重(kg)を入力してください: 65
BMI: 22.5
判定: 普通体重

- `float()` で小数入力に対応
- `f"{bmi:.1f}"` で小数第1位まで表示

BMI判定プログラム

ai_programming_1_03_06.py

```
8 height_m = height_cm / 100
9 bmi = weight / (height_m**2)
10
11 # BMIの判定
12 if bmi < 18.5:
13     result = "低体重"
14 elif bmi < 25.0:
15     result = "普通体重"
16 elif bmi < 30.0:
17     result = "肥満(1度)"
18 else:
19     result = "肥満(2度以上)"
20
21 print(f"BMI: {bmi:.1f}")
22 print(f"判定: {result}")
```

実行結果

身長 (cm) を入力してください: 170
体重 (kg) を入力してください: 65
BMI: 22.5
判定: 普通体重

- `float()` で小数入力に対応
- `f"{bmi:.1f}"` で小数第1位まで表示

BMI判定の解説

計算式

$$\text{BMI} = \text{体重(kg)} \div \text{身長(m)}^2$$

```
height_m = height_cm / 100  
bmi = weight / (height_m ** 2)
```

判定基準（日本肥満学会）

BMI	判定
18.5未満	低体重
18.5以上25未満	普通体重
25以上30未満	肥満(1度)
30以上	肥満(2度以上)

- `if-elif-else` で複数の範囲を判定
- `f文字列` で整形出力

うるう年判定プログラム

ai_programming_1_03_07.py

```
1  # うるう年を判定するプログラム
2
3  # 年を入力
4  year = int(input("年を入力してください: "))
5
6  # うるう年の判定
7  if year % 400 == 0:
8      print(f"{year}年はうるう年です。")
9  elif year % 100 == 0:
10     print(f"{year}年はうるう年ではありません。")
11 elif year % 4 == 0:
12     print(f"{year}年はうるう年です。")
13 else:
14     print(f"{year}年はうるう年ではありません。")
```

実行結果（うるう年）

年を入力してください: 2024
2024年はうるう年です。

実行結果（平年）

年を入力してください: 1900
1900年はうるう年ではありません。

- % で割り切れるかを判定
- 条件の優先順位に注意
(400→100→4の順)

うるう年判定の解説

■ うるう年の判定ルール（優先順位順）

条件	判定
400で割り切れる	うるう年
100で割り切れる	平年
4で割り切れる	うるう年
上記以外	平年

判定の流れ

2000年 → 400で割り切れる → うるう年
1900年 → 100で割り切れる → 平年
2024年 → 4で割り切れる → うるう年
2023年 → 上記以外 → 平年

`if-elif-else` の評価順序を活用した判定

第3章のまとめ

if文

条件が `True` で
ブロックを実行

if-else

2分岐
必ずどちらかを実行

if-elif-else

多分岐
上から順に評価

論理演算子

`and` `or` `not`
複合条件

ネスト

if文の入れ子
段階的な判定

条件式

`値1 if 条件 else 値2`
1行で分岐

条件文は**プログラムの流れを制御**する最も基本的な仕組み

次章の予告：第4章 繰り返し

学ぶこと

- `for` 文による繰り返し
- `while` 文による繰り返し
- `range()` 関数の活用

今回の知識との関係

- 条件文と繰り返しを組み合わせた処理
- `while` 文の条件判定に比較演算子を使用

条件文 + **繰り返し** でプログラムの表現力が大幅に向上